

AI Agent Instructions — Full-Stack Web Application Development From Scratch

0. Document Status

This document is the governing instruction set for AI coding agents working on this application.

The primary development tools are:

- Local development: VS Code and/or Cursor IDE
- Human-assisted coding: Claude Code Pro
- Source control: GitHub
- Frontend: React + TypeScript
- Backend: Node.js + TypeScript
- Hosting: Railway
- Database: PostgreSQL
- Authentication: email/mobile/password plus supported OAuth/OIDC providers
- Payments: Razorpay initially, with Stripe architecture kept extensible
- Security standard: OWASP Top 10:2025 + OWASP ASVS 5.0
- Deployment model: local → GitHub → validation/security → human approval → test/staging → production

The agent must treat this document as a higher-priority project rule than convenience, speed, or assumptions.

1. Core Operating Principle

Do not attempt to build the entire application in one pass.

The application must be developed incrementally as a sequence of small, testable, reviewable changes.

Every meaningful change must follow this lifecycle:

1. Understand the requested feature.
2. Inspect the existing repository.
3. Identify relevant architecture and dependencies.
4. State assumptions explicitly.
5. Propose the implementation approach.
6. Implement only the approved scope.
7. Run automated tests and static checks.
8. Run security checks relevant to the change.
9. Produce a human-readable validation report.

10. STOP and wait for human approval when the change requires human review.
11. Only after approval, make the next controlled change.
12. Never silently expand scope.

The agent must not interpret "build feature X" as permission to redesign unrelated parts of the system.

2. Never Guess Product Requirements

If the business requirement is not defined, the agent must not invent business behavior and permanently encode it.

Examples of things that must not be guessed:

- user roles
- pricing
- subscription rules
- refund rules
- payment business logic
- eligibility criteria
- employer/recruiter behavior
- candidate behavior
- admin privileges
- data-retention periods
- notification policies
- legal/compliance behavior
- required third-party permissions

The agent may propose sensible defaults, but must clearly label them as assumptions.

Use:

ASSUMPTION: ...

Use:

OPEN DECISION: ...

Use:

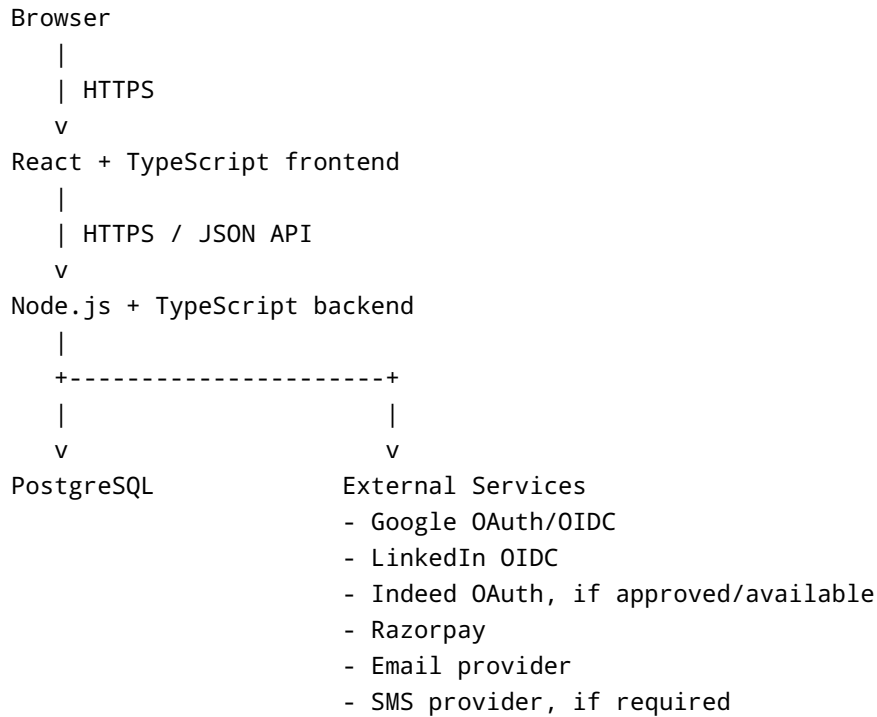
BLOCKED: required information is missing

Do not hide uncertainty inside code.

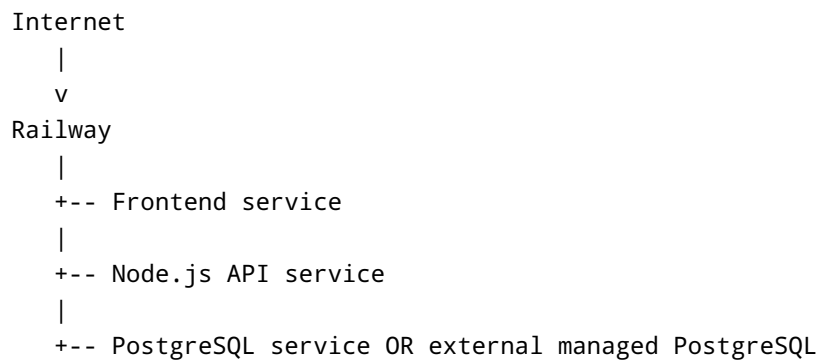
3. Recommended Initial Architecture

Use a clean separation between frontend, backend, database, external integrations, and infrastructure.

Recommended high-level architecture:



Production hosting:



Keep the backend and frontend logically independent even if deployment later places them in the same Railway project.

4. Frontend Requirements

Use:

- React
- TypeScript
- modern React component architecture
- strict TypeScript configuration
- environment-specific configuration
- reusable components
- accessible HTML
- responsive design
- client-side validation where appropriate
- server-side validation always

The frontend must never be treated as a trusted security boundary.

Anything sent by the browser must be considered attacker-controlled.

Never rely on:

```
disabled button  
hidden UI  
frontend role checks  
frontend validation  
frontend pricing calculation  
frontend permission logic
```

for security.

The backend must independently validate authorization, input, state transitions, and payment status.

5. Backend Requirements

Use:

- Node.js
- TypeScript
- REST API unless there is a documented reason to use another API style
- structured routing
- service layer
- data-access/repository layer
- centralized error handling

- centralized authentication/authorization middleware
- schema validation
- structured logging
- secure configuration management

Recommended conceptual structure:

```
backend/  
  src/  
    config/  
    routes/  
    controllers/  
    services/  
    repositories/  
    middleware/  
    validators/  
    auth/  
    integrations/  
      google/  
      linkedin/  
      indeed/  
      razorpay/  
      stripe/  
    database/  
    utils/  
    errors/  
    app.ts  
    server.ts  
  tests/
```

Do not place payment logic directly inside HTTP route handlers.

Do not put database queries throughout arbitrary controllers.

Do not duplicate authorization logic throughout routes.

6. Database Strategy

Preferred database technology

Use PostgreSQL.

Railway currently provides an official PostgreSQL service that can be deployed directly into a Railway project and exposes standard variables such as `DATABASE_URL`, `PGHOST`, `PGPORT`, `PGUSER`, `PGPASSWORD`, and `PGDATABASE`. Railway also supports private networking between services.

Database option A — Railway PostgreSQL

Use this when simplicity and same-platform deployment are the priority.

Advantages:

- PostgreSQL directly inside Railway
- simple networking
- same infrastructure provider
- easy local/production connection model
- good architectural fit

Important:

Railway's free tier is credit-limited. It provides \$1/month of free resource credit after the initial trial, so "free" should be treated as suitable for experimentation/small workloads rather than guaranteed free production infrastructure.

Database option B — Supabase PostgreSQL

Consider this the preferred free-tier alternative.

Current Supabase free tier includes:

- PostgreSQL
- 500 MB database
- 50,000 MAUs
- social OAuth
- 1 GB file storage
- 5 GB egress

However, free projects can be paused after one week of inactivity and automatic database backups are not included.

Supabase is particularly attractive because it can also provide authentication services, reducing the amount of security-sensitive authentication code that the application itself needs to maintain.

Database decision rule

Unless the product has a reason to avoid external managed services:

`PREFERRED STARTING OPTION = Supabase PostgreSQL + Supabase Auth`

ALTERNATIVE = Railway PostgreSQL + application-managed/auth-service authentication

The agent must not switch between these options silently.

7. Database Rules

Use migrations.

Never manually modify production schema without a migration.

Every schema change must have:

- migration
- rollback strategy where practical
- test coverage
- explanation of data impact

Use foreign keys where appropriate.

Use unique constraints for identities and business invariants.

Use indexes based on actual query patterns.

Never store plaintext passwords.

Never store plaintext payment secrets.

Never store OAuth client secrets in source code.

Never store authentication tokens in the database unless there is a documented security reason.

Use UTC timestamps in the database.

Prefer immutable audit records for security-sensitive actions.

8. Authentication Architecture

Authentication is a security-critical subsystem.

Do not write a custom authentication system unnecessarily.

The system should support:

Local account

- email + password
- mobile number + password
- account verification
- password reset
- account lockout/rate limiting where appropriate
- session management
- logout
- password change
- optional MFA later

The same underlying user account should be able to attach multiple login identities.

Example:

```
User
|
+-- Email/password identity
|
+-- Google identity
|
+-- LinkedIn identity
|
+-- Indeed identity
```

Do not create duplicate users automatically merely because the person logged in through a different provider.

Implement account linking carefully and securely.

9. Password Security

Never implement password hashing manually.

Use a well-reviewed password hashing implementation such as Argon2id through a maintained library.

Requirements:

- strong password hashing
- unique salts
- password strength checks
- rate limiting
- secure password-reset tokens

- short-lived password-reset tokens
- one-time-use password-reset tokens
- no password logging
- no password appearing in API responses
- no password appearing in error messages

Passwords must never be stored as plaintext or reversible encryption.

10. Session and Token Security

Prefer secure, HTTP-only session cookies for browser authentication where appropriate.

Cookies should use:

```
HttpOnly  
Secure  
SameSite
```

with values chosen based on the actual deployment architecture.

Do not put long-lived authentication secrets into:

```
localStorage  
sessionStorage  
URLs  
query strings  
logs  
frontend source code
```

JWTs should not be introduced merely because they are popular.

Choose sessions versus tokens based on the actual architecture.

11. Google Login

Google Sign-In must use Google's current OAuth 2.0/OpenID Connect implementation.

Google explicitly supports OIDC for authentication and requires configuration of the OAuth client and redirect URI.

Required environments:

```
LOCAL
STAGING
PRODUCTION
```

Each environment must use appropriate redirect URIs.

Do not reuse production OAuth credentials for local development unless there is a documented reason.

Never commit:

```
GOOGLE_CLIENT_SECRET
```

to GitHub.

12. LinkedIn Login

Use LinkedIn's current "Sign In with LinkedIn using OpenID Connect" approach.

Do not use old/deprecated LinkedIn authentication implementations.

LinkedIn's current documentation identifies the OIDC implementation as the appropriate Sign In with LinkedIn mechanism.

All LinkedIn credentials must be stored as environment secrets.

13. Indeed Login

Indeed currently documents OAuth 2.0 authentication and a "Log in with Indeed" mechanism through its partner platform.

However, access is subject to Indeed's partner/API model.

Therefore:

```
DO NOT assume Indeed login is automatically available.
```

Before implementing it, verify:

- required Indeed developer/partner access
- application approval
- available scopes
- allowed redirect URIs
- intended use
- production access requirements
- user data available through the permitted scope

If access is unavailable, the agent must leave the provider behind a clean integration interface rather than implementing a fake or unsupported provider.

14. Naukri Login

Do not assume Naukri provides a public consumer authentication provider equivalent to Google or LinkedIn.

The requirement must initially be recorded as:

OPEN DECISION:

Can Naukri provide an authorized Sign-In/SSO integration for this application?

Required verification before implementation:

- official Naukri developer documentation
- business/partner program
- OAuth/OIDC availability
- application approval process
- authentication scopes
- API terms
- production availability

If Naukri does not expose an approved identity provider for this use case:

DO NOT:

- scrape Naukri
- automate Naukri login
- imitate Naukri login pages
- collect Naukri passwords
- ask users for their Naukri credentials

Instead, mark Naukri SSO as unsupported until an official integration is available.

15. Authentication Provider Abstraction

Implement authentication providers behind an abstraction.

Conceptually:

```
AuthProvider
|
+-- GoogleProvider
+-- LinkedInProvider
+-- IndeedProvider
+-- NaukriProvider (only if officially supported)
```

The application should not spread provider-specific code throughout the user-management system.

This allows an unsupported provider to remain disabled without redesigning authentication.

16. Email Authentication

Email login should support:

- registration
- email verification
- login
- logout
- password reset
- password change
- account recovery
- rate limiting

The email provider must be configurable.

Do not hardwire one email provider into business logic.

17. Mobile Authentication

The requirement "mobile + password" is different from "mobile OTP."

Implement these as separate concepts.

Phase 1 can support:

```
Mobile number + password
```

with a verified mobile number if the business requires it.

Do not introduce SMS OTP unless there is a real product requirement.

SMS is not inherently free. Phone authentication providers can incur per-SMS charges; for example, Firebase documents phone verification as a billed service outside its basic free authentication allocation.

If OTP is later required, implement it as a separate authentication capability.

18. Payments Architecture

Design payment integration behind an internal interface.

Example:

```
PaymentProvider
|
+-- Razorpay
+-- Stripe
```

Application business logic should not directly depend on Razorpay-specific fields everywhere.

Use an internal payment model such as:

```
Payment
  id
  userId
  provider
  providerPaymentId
  providerOrderId
  amount
  currency
  status
  createdAt
  updatedAt
```

19. Preferred Payment Provider

For an India-focused launch, use Razorpay as the initial implementation candidate.

Razorpay provides web checkout, server-side order creation, test mode, payment verification, and UPI support. Its current documentation states that UPI Collect was deprecated effective February 28, 2026, so new implementations must use the supported current UPI flows rather than the old collect model.

Never mark a payment successful based only on frontend behavior.

The backend must verify payment authenticity.

Razorpay explicitly documents signature verification for payment callbacks/links.

Implement:

```
create payment/order
  |
  v
customer checkout
  |
  v
provider callback/webhook
  |
  v
server-side signature verification
  |
  v
server-side payment state transition
```

20. Stripe Support

Stripe should remain an optional provider.

Stripe is a strong global payment platform, but current Stripe documentation says India accounts are invite-only and local payment methods are not currently supported for Indian Stripe accounts.

Therefore:

```
INITIAL PAYMENT PROVIDER:
Razorpay
```

SECOND PAYMENT PROVIDER:
Stripe, only if business/account eligibility supports it

Do not build Stripe first simply because the API is popular.

21. Payment Security Rules

Never trust:

```
frontend payment-success page
frontend payment status
client-provided amount
client-provided currency
client-provided order ownership
```

The backend must determine:

- what amount is payable
- which user owns the transaction
- what product/service is being purchased
- whether payment actually succeeded
- whether the payment belongs to the correct order

Payment webhooks must be idempotent.

Repeated webhook delivery must not create duplicate transactions or duplicate entitlements.

22. Railway Deployment

Railway supports:

- Railway-generated domains
- custom domains
- automatic SSL certificates
- private service networking
- separate environments

Railway automatically provisions SSL certificates for public HTTP services, and Railway traffic requires HTTPS/TLS.

Therefore:

No public HTTPS requirement exists during ordinary local development.

Local development may use:

`http://localhost:<port>`

OAuth providers can be configured with localhost redirect URIs where their policies permit it.

Production must use HTTPS.

You do NOT need to buy or manually configure an SSL certificate before starting development.

23. Domain Strategy

Recommended sequence:

Phase 1

Local:

`http://localhost`

Phase 2

Railway generated domain:

`https://<project>.up.railway.app`

Phase 3

Custom production domain:

`https://www.example.com`
`https://api.example.com`

or another architecture approved later.

Railway automatically provisions SSL for custom domains after the domain is correctly configured.

24. Environment Strategy

Use at minimum:

```
local
staging
production
```

Railway supports isolated environments and specifically documents staging environments as a normal workflow.

If the budget initially makes staging undesirable, production-only deployment can be used temporarily, but the codebase must still be environment-aware.

Never mix:

```
development credentials
staging credentials
production credentials
```

25. Environment Variables

Create environment-specific secret configuration.

Example categories:

```
DATABASE_URL
SESSION_SECRET
JWT_SECRET
```

OAuth:

```
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET

LINKEDIN_CLIENT_ID
LINKEDIN_CLIENT_SECRET
```

```
INDEED_CLIENT_ID
INDEED_CLIENT_SECRET

NAUKRI_CLIENT_ID
NAUKRI_CLIENT_SECRET
```

Payments:

```
RAZORPAY_KEY_ID
RAZORPAY_KEY_SECRET

STRIPE_SECRET_KEY
STRIPE_WEBHOOK_SECRET
```

Never commit actual values.

Provide:

```
.env.example
```

containing placeholder names only.

Use a `.gitignore` that excludes:

```
.env
.env.*
!.env.example
```

26. Secrets Policy

The following must never appear in:

- source code
- Git history
- issue comments
- pull requests
- screenshots
- logs
- test fixtures
- frontend JavaScript bundles

- database records unless specifically required

Examples:

- passwords
- API keys
- OAuth secrets
- payment secrets
- database passwords
- session secrets
- private certificates
- production tokens

Claude Code must never request the human to paste production secrets into chat.

Use secret stores/environment variables instead.

27. GitHub Repository Structure

Recommended structure:

```
project/  
  frontend/  
  backend/  
  database/  
  docs/  
  scripts/  
  tests/  
  .github/  
    workflows/  
  .claude/  
  .env.example  
  README.md  
  SECURITY.md  
  CONTRIBUTING.md
```

The exact structure may change after architecture review, but changes must be deliberate.

28. Git Branching

Use:

```
main
develop
feature/*
fix/*
security/*
```

or a simpler equivalent.

Do not develop directly on production/main unless explicitly approved.

A feature branch should represent one meaningful change.

Example:

```
feature/user-registration
feature/google-login
feature/payment-order
fix/password-reset-rate-limit
security/session-cookie-hardening
```

29. Pull Request Rules

Every PR must contain:

```
What changed
Why it changed
Files/components changed
Database changes
API changes
Security implications
Tests added/updated
Known limitations
Manual test steps
Rollback considerations
```

A PR must not be approved solely because the application "looks like it works."

30. Claude Code Workflow

Claude Code must work as an engineering agent, not an autonomous owner of the product.

For each task:

1. Read project instructions.
2. Inspect repository.
3. Inspect current architecture.
4. Inspect related tests.
5. Identify assumptions.
6. Implement the smallest safe change.
7. Run validation.
8. Report findings.
9. Stop at approval boundaries.

The agent must not:

- deploy production without explicit approval
- merge production changes without approval
- rotate credentials without authorization
- delete production data
- rewrite the architecture without approval
- introduce a new external service without approval
- add a dependency without explaining why
- disable security controls to make tests pass

31. Claude Code Security Review

Use Claude Code's security review capability as an additional security layer, not as the only security control.

Anthropic's current Claude Code security tooling includes `/security-review`, and Anthropic also provides a GitHub Action for AI-assisted security review of pull requests.

Claude's own documentation warns that automated AI security workflows must be treated carefully around prompt injection and repository permissions. Keep workflow permissions minimal and do not expose secrets unnecessarily.

Recommended workflow:

Developer change

|

```
      v
Claude Code validation
      |
      v
Automated tests
      |
      v
Static analysis
      |
      v
Dependency audit
      |
      v
OWASP/security review
      |
      v
Human review
      |
      v
Human approval
      |
      v
Deployment
```

32. Required Security Baseline

The project must use OWASP Top 10:2025 as a security baseline.

OWASP's current 2025 categories include:

```
A01 Broken Access Control
A02 Security Misconfiguration
A03 Software Supply Chain Failures
A04 Cryptographic Failures
A05 Injection
A06 Insecure Design
A07 Authentication Failures
A08 Software or Data Integrity Failures
A09 Security Logging and Alerting Failures
A10 Mishandling of Exceptional Conditions
```

Also use OWASP ASVS 5.0 as the deeper application-security verification framework. OWASP identifies ASVS 5.0.0 as its current stable version.

33. Security Validation Checklist

Before every production deployment, evaluate at minimum:

Authentication

- ☐ Passwords are securely hashed
- ☐ No plaintext credentials exist
- ☐ Sessions are securely managed
- ☐ OAuth state/nonce protections are correctly implemented
- ☐ Redirect URIs are validated/configured
- ☐ Password-reset tokens are short-lived
- ☐ Login endpoints are rate-limited
- ☐ Account enumeration risk is considered
- ☐ Account-linking logic is secure

Authorization

- ☐ Every protected API endpoint verifies authorization
- ☐ Users cannot access another user's records by changing an ID
- ☐ Role checks occur on the backend
- ☐ Admin endpoints are protected independently
- ☐ Object-level authorization is tested

Input handling

- ☐ Request bodies are schema validated
- ☐ Query parameters are validated
- ☐ IDs are validated
- ☐ File uploads, if present, are restricted
- ☐ SQL injection is prevented
- ☐ Command injection is prevented
- ☐ XSS protections are present
- ☐ SSRF risks are considered

Browser/session security

- ☐ HTTPS is enforced in production
- ☐ Cookies use appropriate security attributes
- ☐ CORS is explicitly configured
- ☐ CSRF protection is considered for cookie-based authentication
- ☐ Security headers are configured
- ☐ Sensitive data is not exposed in URLs

API security

- ☐ Rate limits exist
- ☐ Request size limits exist
- ☐ Authentication errors do not leak sensitive details
- ☐ Production stack traces are not exposed
- ☐ Sensitive logs are redacted

Database

- ☐ Least-privilege database access is used
- ☐ Production database credentials are secret
- ☐ Database migrations are tracked
- ☐ Backups are considered
- ☐ Destructive migrations require approval

Dependency/security supply chain

- ☐ Dependencies are audited
- ☐ Lock files are committed
- ☐ Known critical vulnerabilities are reviewed
- ☐ Unnecessary dependencies are removed
- ☐ Package scripts are reviewed before installing dependencies

Payments

- ☐ Server verifies payment status
- ☐ Payment signatures/webhooks are verified
- ☐ Webhooks are idempotent
- ☐ Client cannot choose arbitrary payment amounts
- ☐ Refund behavior is explicitly defined

34. Testing Strategy

Testing must happen at several levels.

Unit tests

Test:

- services
- business rules
- validators
- utility functions

- authorization decisions

Integration tests

Test:

- API + database
- authentication flows
- payment workflows
- webhook processing
- authorization

End-to-end tests

Test critical user journeys:

```
register  
login  
logout  
password reset  
Google login  
LinkedIn login  
payment  
payment failure  
payment webhook  
role restrictions
```

Security tests

Test:

```
unauthorized API access  
IDOR  
privilege escalation  
SQL injection  
XSS  
CSRF where applicable  
rate-limit bypass  
invalid OAuth state  
invalid payment signature  
replay of payment webhook
```

35. CI Pipeline

Set up GitHub Actions before the application becomes large.

At minimum:

```
install dependencies
typecheck
lint
unit tests
integration tests
build frontend
build backend
dependency/security audit
```

Recommended additional checks:

```
secret scanning
dependency vulnerability scanning
SAST
security review
```

A pull request should fail if critical automated validation fails.

36. Human Approval Gate

The human owner must remain the final authority.

The agent must produce an explicit report:

```
VALIDATION RESULT
```

```
Build:
PASS / FAIL
```

```
TypeScript:
PASS / FAIL
```

```
Lint:
PASS / FAIL
```

Unit tests:
PASS / FAIL

Integration tests:
PASS / FAIL

E2E:
PASS / FAIL / NOT APPLICABLE

Security review:
PASS / FAIL / WARNINGS

Dependency audit:
PASS / FAIL

Database migration:
YES / NO

Production impact:
LOW / MEDIUM / HIGH

Known issues:
...

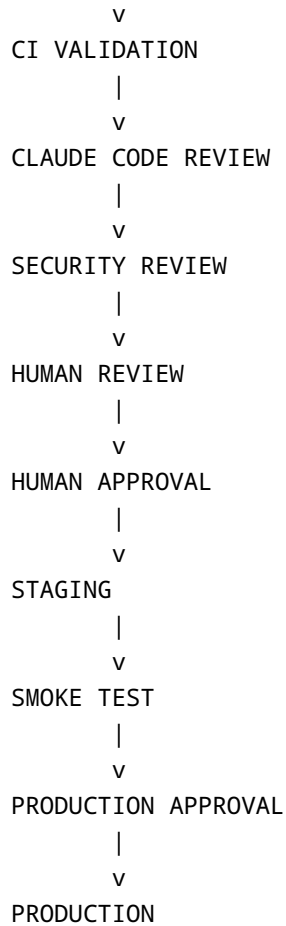
Recommended action:
APPROVE / FIX FIRST / DO NOT DEPLOY

The agent must stop when human approval is required.

37. Deployment Workflow

Preferred workflow:

```
LOCAL DEVELOPMENT
  |
  v
FEATURE BRANCH
  |
  v
LOCAL VALIDATION
  |
  v
PUSH TO GITHUB
  |
```



For the initial prototype, a direct production deployment may be temporarily acceptable, but the repository and configuration must still be designed to support staging.

38. Railway Environment Rules

Use isolated variables per environment.

For example:

```
production DATABASE_URL
staging DATABASE_URL
local DATABASE_URL
```

Do not accidentally connect local development tools to production.

Do not run destructive database commands against production without an explicit confirmation.

Do not use production payment credentials for development testing.

Use payment provider test/sandbox credentials in development and staging.

39. Database Backup Policy

A production application must have a backup strategy before it is considered production-ready.

Do not assume that "database hosted on Railway" automatically means "my data is fully backed up."

The deployment plan must explicitly document:

```
backup frequency
retention
restore procedure
who can restore
how restore is tested
```

For an early prototype, this can be manual.

For a real production system, it should become automated and monitored.

40. Logging and Monitoring

Implement structured application logging.

Never log:

```
passwords
session tokens
OAuth secrets
payment secrets
database passwords
full authentication headers
```

Log useful security events such as:

```
login success/failure
password reset request
```

```
password change
account logout
OAuth login
account linking
payment creation
payment success/failure
webhook receipt
authorization denial
admin actions
```

Logs should include correlation/request IDs where appropriate.

41. Error Handling

Do not expose internal exceptions directly to users.

Development may display detailed diagnostics.

Production should return safe error responses such as:

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "The request could not be processed."
  }
}
```

Do not return:

```
database stack traces
SQL queries
filesystem paths
secret names
OAuth client secrets
internal service details
```

42. API Design Rules

Use consistent API conventions.

For example:

```
POST    /api/v1/auth/register
POST    /api/v1/auth/login
POST    /api/v1/auth/logout
POST    /api/v1/auth/forgot-password
POST    /api/v1/auth/reset-password

GET     /api/v1/users/me

POST    /api/v1/payments/orders
POST    /api/v1/payments/webhooks/razorpay
```

The exact endpoint design can evolve, but API naming must remain consistent.

Version APIs if future compatibility requires it.

43. Documentation Requirements

The repository must contain:

```
README.md
ARCHITECTURE.md
SECURITY.md
DEPLOYMENT.md
DATABASE.md
AUTHENTICATION.md
PAYMENTS.md
ENVIRONMENT.md
```

These documents must be updated when architecture changes.

A future AI agent should be able to understand the project by reading the documentation before editing code.

44. Required Initial Files for AI Agents

Create:

```
AGENTS.md
```

or the equivalent project-level instructions used by the selected coding agent.

It should point the agent to:

```
ARCHITECTURE.md
SECURITY.md
DATABASE.md
AUTHENTICATION.md
PAYMENTS.md
DEPLOYMENT.md
```

The agent must read these files before making architectural changes.

45. AI Agent Safety Rules

The coding agent must refuse to perform the following without explicit human authorization:

- delete production database
- truncate tables
- rotate production secrets
- change authentication provider
- change payment provider
- disable security controls
- disable validation
- bypass tests
- force-push protected branches
- merge to production
- deploy production
- expose secrets
- commit `.env`
- weaken password/security requirements

46. Change Management

Every architectural change must answer:

```
What problem are we solving?
```


Why is the existing design insufficient?

What alternatives were considered?

What are the security implications?

What are the operational implications?

What is the rollback strategy?

What additional cost does this introduce?

Does this change affect existing users or data?

Do not introduce architecture simply because a framework/library makes implementation convenient.

47. Initial Technology Decision

Unless subsequent requirements contradict it, start with this architecture:

Frontend:
React + TypeScript

Backend:
Node.js + TypeScript

Database:
PostgreSQL

Initial database/auth candidate:
Supabase PostgreSQL + Supabase Auth

Hosting:
Railway

Authentication:
Email/password
Mobile/password
Google OIDC
LinkedIn OIDC
Indeed only after official access is confirmed
Naukri only after official access is confirmed

Payments:

Razorpay first
Stripe behind provider abstraction

Source control:
GitHub

Development:
VS Code / Cursor

AI development:
Claude Code Pro

Testing:
Vitest/Jest + integration tests + Playwright or equivalent E2E

Security:
OWASP Top 10:2025
OWASP ASVS 5.0

CI:
GitHub Actions

48. What the Agent Must Do Before Writing Application Code

Before implementing the first feature:

Step 1 — Repository

Create the GitHub repository.

Step 2 — Project skeleton

Create frontend/backend structure.

Step 3 — TypeScript

Enable strict TypeScript configuration.

Step 4 — Environment model

Create:

```
.env.example
```

and document all environment variables.

Step 5 — Database decision

Confirm:

```
Railway PostgreSQL  
OR  
Supabase PostgreSQL
```

Do not silently choose one.

Step 6 — Authentication decision

Confirm:

```
Supabase Auth  
OR  
custom Node authentication  
OR  
another managed authentication provider
```

Step 7 — OAuth verification

Verify actual access for:

```
Google  
LinkedIn  
Indeed  
Naukri
```

Do not implement providers based on assumptions.

Step 8 — Payment account verification

Confirm:

```
Razorpay account availability  
business/KYC requirements
```

```
test credentials
webhook availability
```

Step 9 — Domain strategy

Decide whether the first deployed environment uses:

```
Railway domain
```

or a purchased/custom domain.

Step 10 — Deployment architecture

Decide:

```
production only
```

or:

```
staging + production
```

Step 11 — Security baseline

Create the security checklist.

Step 12 — CI

Create GitHub Actions.

Only after these steps should feature implementation begin.

49. First Development Milestone

Do NOT start with payments or SSO.

The first milestone should be:

```
Application skeleton
+
frontend
```

```
+  
backend  
+  
database connectivity  
+  
health check  
+  
environment configuration  
+  
basic CI  
+  
security baseline
```

Expected successful result:

```
React application loads locally.  
Node API starts locally.  
API health endpoint works.  
Frontend can call backend.  
Backend can connect to PostgreSQL.  
Database migration works.  
Tests run.  
CI runs.  
No secrets are committed.  
Production build succeeds.
```

Only after this milestone passes should user authentication be implemented.

50. Authentication Milestone

Implement in this order:

1. User model
2. Email/password registration
3. Email verification
4. Login
5. Logout
6. Session management
7. Password reset
8. Mobile/password
9. Google login
10. LinkedIn login
11. Indeed login, if officially approved

12. Naukri login, if officially supported
13. Account linking
14. Security hardening

Do not implement all providers simultaneously.

51. Payments Milestone

Implement in this order:

1. Product/pricing model
2. Internal payment abstraction
3. Razorpay test credentials
4. Server-side order creation
5. Frontend checkout
6. Payment signature verification
7. Webhooks
8. Idempotency
9. Payment status reconciliation
10. Refund/cancellation model
11. Production credentials

52. Production Readiness Gate

The application is NOT production-ready simply because it works.

Before production, verify:

Build passes
Tests pass
Security review passes
Dependency audit passes
Secrets are externalized
HTTPS works
OAuth redirects work
Authentication works
Authorization works
Payment verification works
Webhook handling works
Database backup strategy exists

```
Logging exists
Monitoring exists
Error handling is safe
Rate limits exist
CORS is restricted
Production environment variables are correct
Rollback procedure exists
```

53. Final Agent Rule

The agent must optimize for:

```
correctness
security
maintainability
observability
testability
controlled change
```

not:

```
maximum code produced
minimum number of files
fastest possible implementation
```

The agent should prefer a small, correct implementation over a large speculative implementation.

When uncertain, explain the uncertainty.

When a requirement is unsupported by a third party, say so.

When a security issue is discovered, stop and report it.

When human approval is required, stop and wait.

Never conceal uncertainty merely to keep development moving.

End of Agent Instructions